Clasificación Automatizada de Residuos Urbanos mediante Redes Convolucionales en PyTorch

Práctica 1: Aprendizaje Automático II
Grado en Ciencia e Ingeniería de Datos — Curso 2026/2027

Autores:
Abel Castro Villar
Lucas Rodríguez Hernández

Asignatura:
Aprendizaje Automático II
Universidad de Las Palmas de Gran Canaria

Resumen

En esta práctica se desarrolló en PyTorch un clasificador de imágenes de residuos en siete categorías a partir de 2.340 imágenes. Se compararon cinco configuraciones manuales de una CNN propia, se exploró Optuna y se evaluaron ResNet-18 y MobileNetV2 con transferencia de aprendizaje. La quinta CNN alcanzó un 75,14 % de accuracy en test. En la ejecución definitiva, MobileNetV2 obtuvo el mejor resultado: **95,53 %** (342 de 358 imágenes), con un recall de **91,49 %** para metal; ResNet-18 alcanzó un 88,83 %. La parada anticipada se aplicó en todas las fases. El informe describe las decisiones, resultados y errores observados, usando las métricas de la última ejecución guardada en el cuaderno.

Octubre de 2026

Índice

1. Introducción y Planteamiento del Problema **2**

2. Conjunto de Datos y Preprocesamiento Avanzado **2**
2.1. Partición y Balance de Clases . . . 2
2.2. Carga Segura de Imágenes: `SafeImageFolder` . . . 3
2.3. Pipeline de Aumento de Datos (*Data Augmentation*) . . . 3

3. Arquitectura Convolucional Propia y Control del Sobreajuste **3**
3.1. Definición Modular de `WasteCNN` . . . 4
3.2. Parada Anticipada (*Early Stopping*) . . . 4

4. Búsqueda Manual de Hiperparámetros y Diagnóstico del Metal **4**
4.1. Prueba 1: Modelo Base (96 × 96, 3 Convs, FC 128, Batch 32) — Accuracy: 69,00 % 4
4.2. Diagnóstico Técnico del «Problema del Metal» . . . 5
4.3. Prueba 2: Hipótesis de Capacidad Densa (FC 256) — Accuracy: 61,00 % . . . 5
4.4. Pruebas 3 y 4: Hipótesis de Resolución y Regularización — Acc: 66,00 % y 67,00 % 6
4.5. Prueba 5: Más profundidad, filtros y tamaño de lote . . . 6
4.6. Síntesis de las cinco pruebas manuales . . . 7

5. Búsqueda Automática de Hiperparámetros (Optuna) **7**

6. Transferencia de Conocimiento y Comparativa de Arquitecturas **8**
6.1. ResNet-18 . . . 8
6.2. MobileNetV2 . . . 8

7. Resultados y discusión **9**
7.1. Comparación de matrices de confusión . . . 9

1 Introducción y Planteamiento del Problema

La clasificación y valorización de residuos sólidos urbanos e industriales representa uno de los retos operacionales más acuciantes de la economía circular contemporánea. En las plantas de tratamiento convencionales, las líneas de triaje dependen en gran medida del trabajo manual de operarios, una labor expuesta a riesgos biológicos, mecánicos y químicos, además de caracterizarse por un rendimiento variable y elevados costes. La automatización de estos procesos mediante visión artificial y aprendizaje profundo ofrece la capacidad de inspeccionar flujos continuos de residuos a alta velocidad de manera estandarizada y segura.

El objetivo de esta práctica es concebir, entrenar y validar un modelo de aprendizaje automático supervisado en PyTorch [1] capaz de clasificar imágenes de residuos en 7 categorías:

1. **Cartón (`t-cardboard`):** Cajas, planchas de embalaje y paquetes de celulosa seca.
2. **Residuos Electrónicos (`t-e-waste`):** Pequeños electrodomésticos, periféricos de ordenador, cables y placas de circuitos.
3. **Vidrio (`t-glass`):** Botellas transparentes y coloreadas, frascos y fragmentos vítreos.
4. **Metal (`t-metal`):** Latas de bebidas, envases de conservas, aerosoles y residuos metálicos férricos y de aluminio.
5. **Orgánico (`t-organic`):** Frutas, verduras, restos de comida y residuos biodegradables.
6. **Plástico (`t-plastic`):** Botellas de polietileno (PET), envases domésticos, envoltorios y bolsas.
7. **Textil (`t-textile`):** Prendas de ropa desechadas, retales y tejidos sintéticos y naturales.

Para superar con solvencia la práctica y alcanzar la máxima calificación, el trabajo se adhiere de manera rigurosa a las directrices docentes fijadas:

- Construir una arquitectura convolucional combinando convolución, *pooling*, *dropout* y parada anticipada (*Early Stopping*) para mitigar el sobreajuste.
- Seleccionar un conjunto de datos real (evitando problemas triviales como MNIST o CIFAR) con al menos 50 imágenes por clase y entre 5 y 15 categorías.
- Alcanzar una tasa de acierto (*accuracy*) en el conjunto de test de al menos el 85 %.
- Documentar un proceso de búsqueda de hiperparámetros manual con un mínimo de 5 configuraciones experimentales detalladas, explorando todos los hiperparámetros sugeridos (épocas, tasa de aprendizaje, tamaño del lote, número de capas convolucionales y densas, filtros por capa, neuronas FC y funciones de activación).
- Implementar los elementos opcionales: aumento de datos (*Data Augmentation*), afinado automático (*Tuning*) y transferencia de conocimiento (*Transfer Learning*) con arquitecturas alternativas.
- Entregar el código fuente final estrictamente limpio y libre de comentarios (`#`), acompañándolo de una memoria formal con curvas de aprendizaje y matrices de confusión.

2 Conjunto de Datos y Preprocesamiento Avanzado

2.1 Partición y Balance de Clases

Para garantizar que la evaluación de los modelos sea insesgada y representativa, se realizó una partición estratificada mediante la herramienta especializada `split-folders`. Se fijaron proporciones del **70 % para Entrenamiento (1.634 imágenes)**, **15 % para Validación (348 imágenes)** y **15 % para Test (358 imágenes)**. La Tabla 1 detalla el desglose por categorías.

Tabla 1: Distribución estratificada de imágenes por clase en el dataset de residuos.

Categoría de Residuo	**Train (70 %)**	**Val (15 %)**	**Test (15 %)**	**Total**
`t-cardboard` (Cartón)	246	52	54	352
`t-e-waste` (Electrónico)	292	62	64	418
`t-glass` (Vidrio)	226	48	49	323
`t-metal` (Metal)	214	46	47	307
`t-organic` (Orgánico)	256	55	56	367
`t-plastic` (Plástico)	216	46	47	309
`t-textile` (Textil)	184	39	41	264
Total Muestras	**1.634**	**348**	**358**	**2.340**

Todas las categorías superan ampliamente el mínimo docente de 50 muestras (oscilando entre 264 y 418 imágenes por clase), ofreciendo un soporte estadístico sólido para el entrenamiento y la evaluación.

2.2 Carga Segura de Imágenes: `SafeImageFolder`

Durante la lectura masiva de archivos en sistemas heterogéneos (Windows, Google Drive y Linux), es habitual la generación fortuita de metadatos del explorador (como `.DS_Store` o `Thumbs.db`) que provocan la caída de `torchvision.datasets.ImageFolder` al interpretar carpetas vacías o archivos no compatibles. Se programó la clase robusta `SafeImageFolder`, la cual sobrescribe el método `find_classes` para filtrar e indexar únicamente aquellas carpetas que contengan extensiones de imagen reconocibles (`.jpg`, `.png`, `.jpeg`, `.webp`).

2.3 Pipeline de Aumento de Datos (*Data Augmentation*)

Para expandir artificialmente la variabilidad visual del conjunto de entrenamiento y enseñar a la red invariancia frente a la perspectiva y la iluminación, se programó un pipeline estocástico *on-the-fly* que se aplica en memoria en cada época:

- **Volteos horizontal y vertical:** `RandomHorizontalFlip(p=0.5)` y `RandomVerticalFlip(p=0.5)`. En un clasificador de basura en cinta, un objeto carece de orientación espacial preferente.
- **Rotaciones controladas:** `RandomRotation(degrees=15)`, simulando pequeñas desalineaciones en la perspectiva de captura.
- **Alteración cromática (*ColorJitter*):** Fluctuación aleatoria de brillo ($\pm 30\,\%$), contraste ($\pm 30\,\%$), saturación ($\pm 30\,\%$) y matiz ($\pm 5\,\%$) para evitar que el modelo sobreajuste tonalidades de luz concretas.
- **Transformaciones afines (*RandomAffine*):** Traslaciones relativas del $10\,\%$ y variaciones de escala entre $0,9\times$ y $1,1\times$.
- **Normalización ImageNet:** Sustracción de media $\boldsymbol{\mu} = [0,485, 0,456, 0,406]$ y división por desviación $\boldsymbol{\sigma} = [0,229, 0,224, 0,225]$.

En validación y test se prescinde de todo aumento estocástico, aplicando exclusivamente redimensionado y normalización determinista.

3 Arquitectura Convolucional Propia y Control del Sobreajuste

3.1 Definición Modular de `WasteCNN`

Para satisfacer el objetivo básico de diseñar una CNN desde cero en PyTorch, se implementó la clase modular y parametrizable `WasteCNN`. La arquitectura sigue un esquema piramidal compuesto por L bloques de extracción de características ($L = 3$ en las Pruebas 1–4 y $L = 4$ en la Prueba 5) seguidos de un clasificador completamente conectado:

$$\mathbf{X}_k = \text{MaxPool2d}\Big(\sigma\big(\text{Conv2d}_{C_{k-1}\to C_k}^{(K_k,p_k)}(\mathbf{X}_{k-1})\big)\Big), \quad k \in \{1,\dots,L\} \tag{1}$$

donde $\sigma(\cdot)$ representa la función de activación no lineal (ReLU o LeakyReLU). En la configuración de 3 etapas se emplean filtros $(16, 32, 64)$ con kernel 5×5 y padding 2; en la configuración profunda de 4 etapas se emplean filtros $(32, 64, 128, 256)$ con kernel 3×3 y padding 1. Tras L etapas de submuestreo con ventana 2×2 y paso 2, las dimensiones espaciales de la entrada se reducen en un factor 2^L. La dimensión del vector aplanado se calcula dinámicamente:

$$D_{\text{flat}} = C_L \times \left(\frac{H_{\text{in}}}{2^L}\right) \times \left(\frac{W_{\text{in}}}{2^L}\right) \tag{2}$$

El vector resultante se proyecta mediante dos capas lineales completamente conectadas con regularización estocástica por dropout antes y después de la capa oculta:

$$\mathbf{h} = \text{Dropout}_p\Big(\sigma\big(\mathbf{W}_{\text{fc1}} \cdot \text{Dropout}_p(\mathbf{x}_{\text{flat}}) + \mathbf{b}_{\text{fc1}}\big)\Big) \tag{3}$$

$$\hat{\mathbf{y}} = \mathbf{W}_{\text{fc2}}\mathbf{h} + \mathbf{b}_{\text{fc2}}, \quad \mathbf{W}_{\text{fc2}} \in \mathbb{R}^{7\times N_{\text{fc}}} \tag{4}$$

3.2 Parada Anticipada (*Early Stopping*)

Para evitar que el modelo continúe entrenando una vez alcanzado el mínimo del error de generalización, se implementó la clase desacoplada `EarlyStopping`. El algoritmo supervisa la pérdida en el conjunto de validación (*Validation Cross-Entropy Loss*):

$$\mathcal{L}_{\text{val}} = -\frac{1}{M}\sum_{i=1}^{M}\sum_{c=1}^{7} y_{i,c}\log \hat{p}_{i,c} \tag{5}$$

Cada vez que $\mathcal{L}_{\text{val}}$ mejora respecto al mínimo histórico registrado, se realiza una copia profunda en memoria de los pesos del modelo (`copy.deepcopy(model.state_dict())`) y se reinicia el contador de paciencia. Si durante $P = 5$ épocas consecutivas no se produce una mejora estricta, el entrenamiento se detiene y se restauran automáticamente los pesos óptimos de la mejor época, descartando los estados posteriores degradados por sobreajuste.

4 Búsqueda Manual de Hiperparámetros y Diagnóstico del Metal

El proceso experimental se inició con una exploración manual sistemática de **cinco configuraciones diferentes** sobre la red propia `WasteCNN`, variando de manera controlada todos los hiperparámetros listados en las instrucciones de la práctica: número de épocas, tasa de aprendizaje, tamaño del lote (*batch size*), número de capas convolucionales, número de filtros por capa, número de neuronas en la capa completamente conectada y funciones de activación.

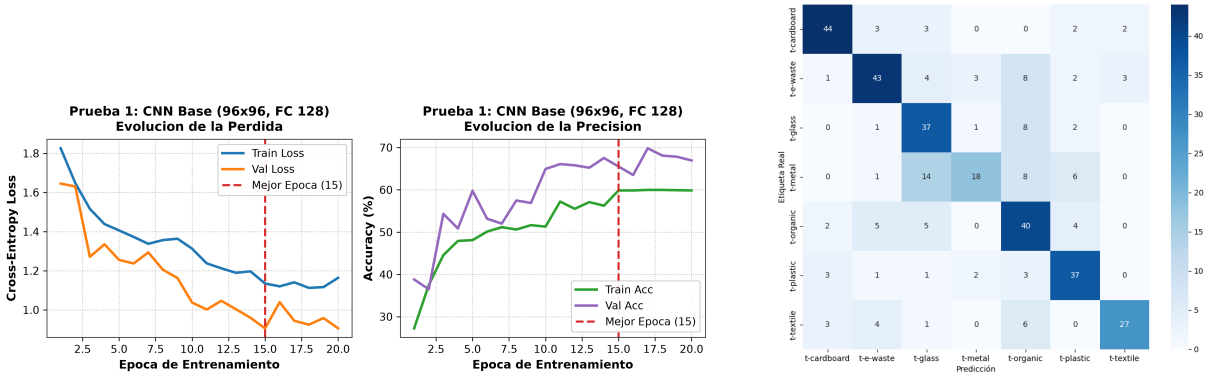
4.1 Prueba 1: Modelo Base (96×96, 3 Convs, FC 128, Batch 32) — Accuracy: 69,00 %

Configuración: Entrada 96×96, 3 bloques convolucionales (16, 32, 64 filtros de 5×5), clasificador de 2 capas lineales con $N_{\text{fc}} = 128$ neuronas ocultas, activación ReLU, Dropout $p = 0,25$, optimizador Adam con $\eta = 0,001$ y tamaño de lote $B = 32$.

Comportamiento: El entrenamiento se prolongó durante 20 épocas, activándose el *Early Stopping* en la época 20 y rescatando el punto óptimo alcanzado en la **época 15** con una pérdida de validación de $\mathcal{L}_{\text{val}} = 0,9056$ (Figura 1a).

Diagnóstico de la Matriz de Confusión Inicial: Al evaluar el modelo en el conjunto de test independiente (358 imágenes), el accuracy global se situó en el 69,00 %. Sin embargo, la inspección analítica de la matriz de confusión (Figura 1b) desveló un fallo estructural crítico:

- La clase `t-metal` obtuvo un recall de tan solo el **38,30 %** (únicamente 18 aciertos de 47 muestras reales).
- Un total de **14 latas y objetos metálicos fueron clasificados erróneamente como vidrio (`t-glass`)**, y 8 como residuo orgánico (`t-organic`).



(a) Pérdida y precisión.

(b) Matriz de confusión.

Figura 1: Evaluación de la Prueba 1 en test: 69,00 % de accuracy. La clase metal obtuvo 18 aciertos de 47; 14 objetos metálicos se clasificaron como vidrio y 8 como orgánico.

4.2 Diagnóstico Técnico del «Problema del Metal»

El colapso selectivo entre metal y cristal responde a tres causas físicas y ópticas fundamentales:

1. **Reflejos especulares idénticos:** Tanto el aluminio pulido de una lata aplastada como la superficie curva de una botella de vidrio generan zonas de saturación lumínica blanca de alta frecuencia (destellos especulares) con gradientes bruscos hacia sombras adyacentes.
2. **Ausencia de color intrínseco en cristal:** El vidrio transparente adopta la textura y el color del suelo donde yace, mimetizando el comportamiento óptico de una lámina metálica que refleja su entorno.
3. **Pérdida de rebordes a baja resolución:** A 96×96 píxeles, la pequeña anilla o arista circular de apertura de una lata se comprime en 1 o 2 píxeles, siendo difuminada por las sucesivas etapas de MaxPool.

4.3 Prueba 2: Hipótesis de Capacidad Densa (FC 256) — Accuracy: 61,00 %

Hipótesis: Se conjeturó que el clasificador completamente conectado no disponía de suficientes neuronas para trazar la frontera de decisión no lineal entre los vectores latentes de metal y vidrio. Se duplicó la capa oculta densa de 128 a **256 neuronas**.

Resultado: El modelo sufrió sobreajuste severo. El accuracy en test cayó 8 puntos porcentuales hasta el **61,00 %**, y el recall de metal se desplomó al **23,40 %** (solo 11 aciertos). El vector aplanado ($64 \times 12 \times 12 = 9{,}216$) conectado a 256 neuronas elevó los parámetros a 2,43 millones, provocando que la capa densa memorizara el ruido espacial del conjunto de entrenamiento sin extraer mejores descriptores visuales (Figura 2).

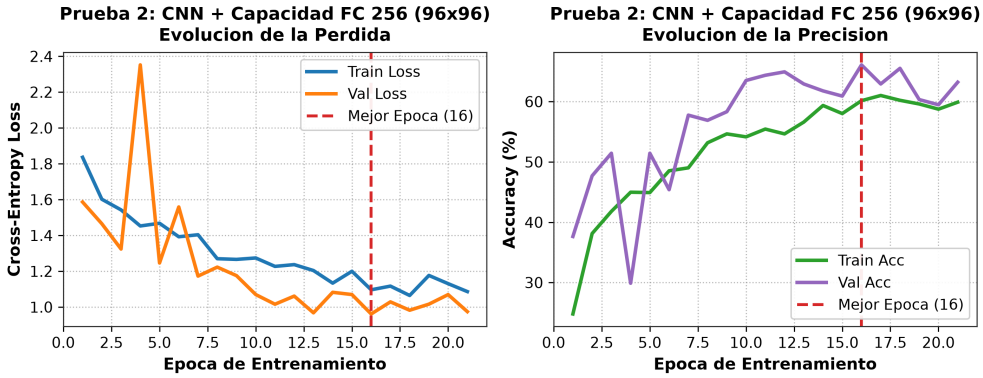


Figura 2: Curvas de aprendizaje de la Prueba 2 (CNN + FC 256). Parada anticipada en época 21 rescatando la época 16 ($\mathcal{L}_{\text{val}} = 0,9613$). Se observa divergencia prematura entre entrenamiento y validación.

4.4 Pruebas 3 y 4: Hipótesis de Resolución y Regularización — Acc: 66,00 % y 67,00 %

Prueba 3 (128×128, FC 128, ReLU, LR 0.001): Se retornó a 128 neuronas densas y se incrementó la resolución de entrada de 96×96 a 128×128 píxeles para preservar las aristas de las latas. El modelo mejoró respecto a la Prueba 2 alcanzando un **66,00 %** en test (mejor época 20, $\mathcal{L}_{\text{val}} = 0,9152$), con un recall en metal del 29,79 %.

Prueba 4 (128×128, LeakyReLU, Dropout 0.40, LR 0.0005): Para mitigar la muerte de neuronas ReLU y estabilizar el descenso de gradiente a mayor resolución, se introdujo la activación **LeakyReLU** ($\alpha = 0,1$), se elevó la regularización Dropout a $p = 0,40$ y se redujo la tasa de aprendizaje a $\eta = 0,0005$. La red entrenó durante 38 épocas y restauró la **época 33** con una pérdida de validación de 0,7707. En test obtuvo **67,00 %** y el recall de metal fue del 38,30 %.

Tabla 2: Comparación de las Pruebas 3 y 4.

Métrica	Prueba 3	Prueba 4
Entrada / bloques	128×128 / 3	128×128 / 3
Activación / dropout	ReLU / 0,25	LeakyReLU / 0,40
Tasa de aprendizaje	0,001	0,0005
Mejor época / Val Loss	20 / 0,9152	33 / 0,7707
Accuracy en test	66,00 %	67,00 %
Recall de metal	29,79 %	38,30 %

4.5 Prueba 5: Más profundidad, filtros y tamaño de lote

Tras las cuatro primeras pruebas se amplió la CNN a cuatro bloques, con filtros $(32, 64, 128, 256)$, entrada de 128×128, lote 64, LeakyReLU, Dropout $p = 0,40$ y Adam con tasa 0,0005. La motivación fue aumentar la capacidad de representar patrones espaciales más amplios y comprobar si un lote mayor estabilizaba el entrenamiento. La red contiene 2,49 millones de parámetros.

En la ejecución definitiva, la parada anticipada seleccionó la época 33 de un máximo de 35, con pérdida de validación 0,6348. El entrenamiento duró 6,10 minutos. En test obtuvo **75,14 %** (269 de 358), mejor resultado de las CNN artesanales, aunque por debajo del 85 % orientativo. El recall de metal fue 48,94 % (23 de 47), y la matriz registrada muestra siete ejemplos de metal confundidos con vidrio. Estos resultados motivaron evaluar modelos preentrenados. El detalle de esta prueba queda recogido en la tabla de síntesis de la Sección 3; se omite el desglose de clasificación por clase para mantener el informe conciso.

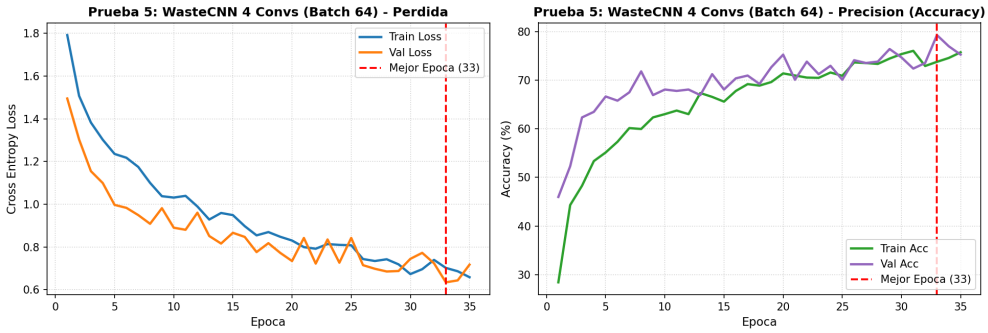


Figura 3: Curvas de aprendizaje de la Prueba 5 (`WasteCNN` profunda de 4 bloques convolucionales $[32, 64, 128, 256]$ y tamaño de lote $B = 64$).

4.6 Síntesis de las cinco pruebas manuales

La Tabla 3 consolida las cinco configuraciones manuales realizadas sobre `WasteCNN`.

Tabla 3: Comparativa cuantitativa de las cinco configuraciones manuales sobre la red propia `WasteCNN`.

Configuración	Capas / Filtros Conv.	Resolución	Batch	FC / Activ.	Drop.	LR	Val Loss	Test Acc.
Prueba 1 (Base)	3 (16, 32, 64)	96×96	32	128 / ReLU	0,25	0,0010	0,9056	69,00 %
Prueba 2 (Capacidad)	3 (16, 32, 64)	96×96	32	256 / ReLU	0,25	0,0010	0,9613	61,00 %
Prueba 3 (Resolución)	3 (16, 32, 64)	128×128	32	128 / ReLU	0,25	0,0010	0,9152	66,00 %
Prueba 4 (Regularizada)	3 (16, 32, 64)	128×128	32	128 / LeakyReLU	0,40	0,0005	**0,7707**	67,00 %
Prueba 5 (Profunda)	**4** (32, 64, 128, 256)	**128×128**	**64**	**128 / LeakyReLU**	**0,40**	**0,0005**	0,6348	**75,14 %**

Conclusión Metodológica de la Fase Manual: La progresión de las cinco pruebas elevó el accuracy de test del 69,00 % al 75,14 %. El aumento de profundidad mejoró la red artesanal, aunque no fue suficiente para alcanzar el 85 % orientativo; por ello se evaluaron también la búsqueda automática y la transferencia de aprendizaje.

5 Búsqueda Automática de Hiperparámetros (Optuna)

Como extensión de la búsqueda manual, se ejecutó Optuna con el sampler TPE y el podador `MedianPruner` sobre la CNN propia de tres bloques y entrada 96×96. Se realizaron diez ensayos, con un máximo de ocho épocas por ensayo. El espacio de búsqueda incluyó la tasa de aprendizaje (10^{-4} a 5×10^{-3}), Dropout (0,15 a 0,50), neuronas de la capa densa (64, 128 o 256) y activación (ReLU o LeakyReLU).

El mejor ensayo registrado obtuvo **66,95 % de accuracy en validación**, con tasa de aprendizaje 0,000433, Dropout 0,50, 64 neuronas y LeakyReLU. El podador detuvo los ensayos cuyo progreso quedó por debajo de la mediana observada. Esta búsqueda ayudó a comparar hiperparámetros con un presupuesto acotado, pero no superó los resultados de las pruebas manuales más profundas ni los modelos preentrenados. El 66,95 % corresponde a validación y no debe confundirse con la evaluación final en test.

Tabla 4: Resultados empíricos de los 10 ensayos de optimización bayesiana con Optuna (TPE) sobre `WasteCNN`.

Ensayo	Tasa de Aprendizaje (η)	Dropout (p)	Neuronas FC	Activación	Comportamiento / Épocas	Mejor Val Acc.
1 (Óptimo)	$4,33 \times 10^{-4}$	0,50	64	LeakyReLU	Completo (8 épocas)	**66,95 %**
2	$2,96 \times 10^{-3}$	0,35	256	LeakyReLU	Completo (8 épocas)	56,03 %
3	$2,00 \times 10^{-4}$	0,20	128	ReLU	Completo (8 épocas)	65,52 %
4	$1,70 \times 10^{-4}$	0,25	256	ReLU	Completo (8 épocas)	62,93 %
5	$1,02 \times 10^{-3}$	0,15	64	ReLU	Completo (8 épocas)	59,48 %
6	$2,36 \times 10^{-3}$	0,25	128	ReLU	Podado (*MedianPruner*, época 3)	36,78 %
7	$1,10 \times 10^{-4}$	0,50	128	ReLU	Podado (*MedianPruner*, época 3)	47,70 %
8	$2,10 \times 10^{-4}$	0,50	128	ReLU	Podado (*MedianPruner*, época 3)	49,43 %
9	$1,40 \times 10^{-4}$	0,20	256	ReLU	Podado (*MedianPruner*, época 4)	53,45 %
10	$4,00 \times 10^{-4}$	0,25	256	ReLU	Podado (*MedianPruner*, época 7)	59,20 %

6 Transferencia de Conocimiento y Comparativa de Arquitecturas

Las cinco pruebas manuales mostraron que la CNN propia mejoraba al aumentar su profundidad, pero su mejor resultado fue 75,14 % en test. Para aprovechar características visuales aprendidas previamente, se adaptaron dos redes preentrenadas con ImageNet. En ambas se reemplazó la capa de salida para producir siete clases y se entrenó con imágenes de 224×224, lote 32 y Adam con tasa de aprendizaje 0,0005.

6.1 ResNet-18

ResNet-18 incorpora conexiones residuales que permiten propagar información a través de una red profunda. En cada bloque, la entrada se suma a la transformación aprendida, $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$. Se sustituyó su capa final de 512 unidades por una salida de siete clases y se ajustaron los pesos del modelo. La parada anticipada seleccionó la época 4 (pérdida de validación 0,2390) y detuvo el entrenamiento en la época 9. En test obtuvo **88,83 %** (318/358), superando el objetivo orientativo del 85 %. El modelo cuenta con 11,18 millones de parámetros y el entrenamiento registrado duró 201,06 segundos.

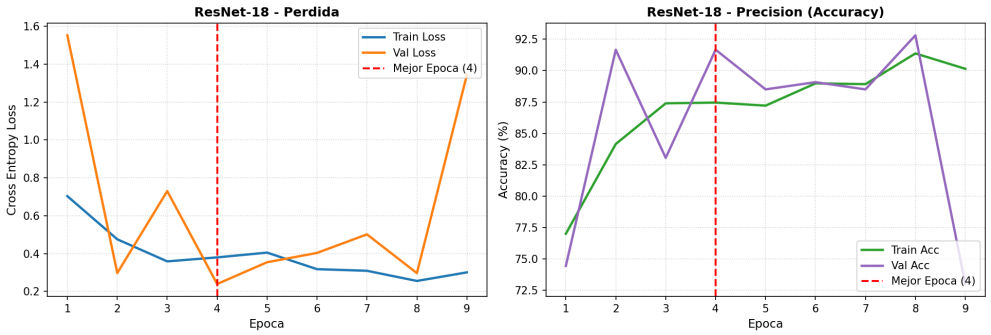


Figura 4: Curvas de pérdida y accuracy de ResNet-18. La línea vertical marca la época seleccionada por validación.

6.2 MobileNetV2

MobileNetV2 emplea bloques residuales invertidos y convoluciones separables en profundidad. Cada bloque expande canales con convoluciones puntuales, realiza el filtrado espacial por canal y proyecta después a una representación compacta; así reduce el coste frente a convoluciones estándar. Se sustituyó su capa clasificadora de 1.280 unidades por una salida de siete clases. Tras cuatro épocas, obtuvo su menor pérdida de validación (0,1709); la parada anticipada se activó en la época 9. El entrenamiento duró 208,73 segundos.

En test, MobileNetV2 alcanzó **95,53 %** (342/358), el mejor resultado de las ejecuciones comparadas. Para la clase metal obtuvo precisión y recall de 91,49 % (43 de 47 ejemplos correctamente identificados). El modelo final tiene 2,23 millones de parámetros tras adaptar la capa clasificadora.

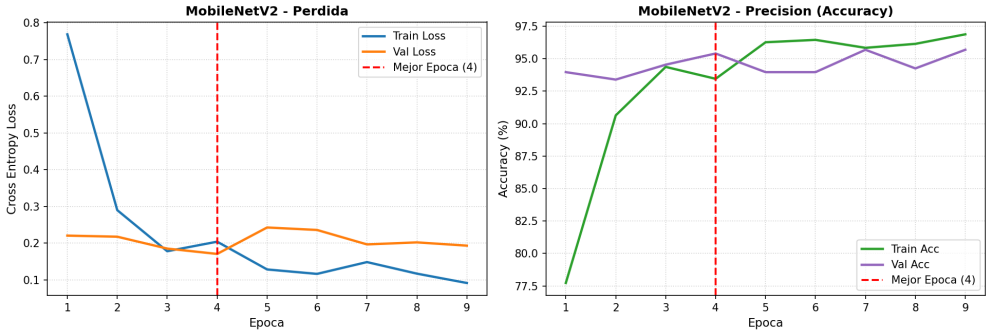


Figura 5: Curvas de pérdida y accuracy de MobileNetV2. La línea vertical marca la época seleccionada por validación.

7 Resultados y discusión

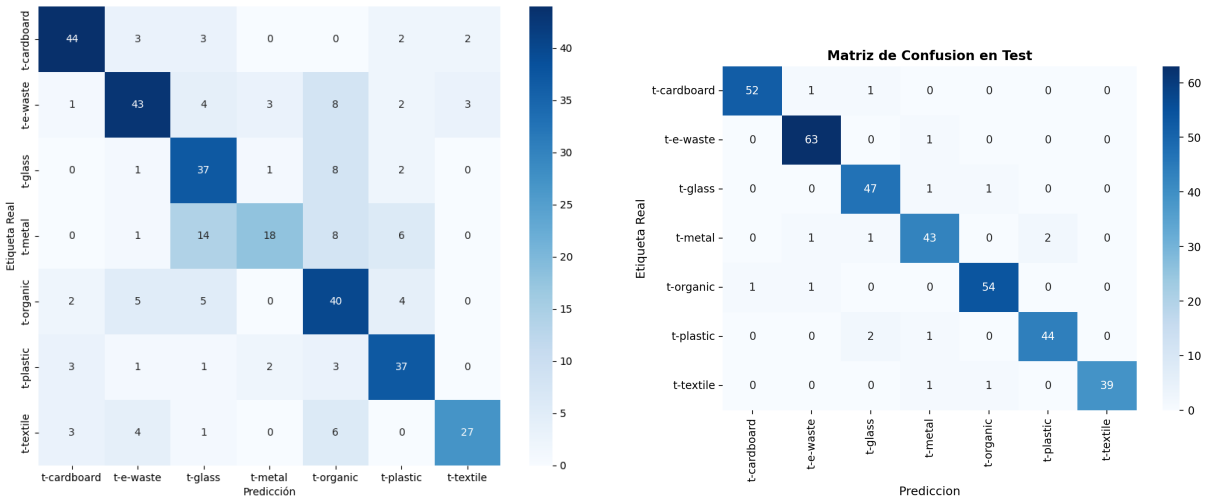
La comparación en test sitúa a MobileNetV2 en primer lugar (95,53 %), seguido de ResNet-18 (88,83 %) y de la CNN artesanal de cuatro bloques (75,14 %). Los dos modelos preentrenados superan el 85 % orientativo. Las métricas por clase de MobileNetV2 muestran un rendimiento equilibrado, con un recall de metal del 91,49 % y un promedio macro de F1 de 0,9545.

Tabla 5: Métricas de clasificación de MobileNetV2 en el conjunto de test.

Categoría	Precisión	Recall	F1	Soporte
Cartón	0,9811	0,9630	0,9720	54
Residuos electrónicos	0,9545	0,9844	0,9692	64
Vidrio	0,9216	0,9592	0,9400	49
Metal	0,9149	0,9149	0,9149	47
Orgánico	0,9643	0,9643	0,9643	56
Plástico	0,9565	0,9362	0,9462	47
Textil	1,0000	0,9512	0,9750	41
Accuracy			0,9553	358
Promedio macro	0,9561	0,9533	0,9545	358
Promedio ponderado	0,9558	0,9553	0,9554	358

7.1 Comparación de matrices de confusión

La Figura 6 contrasta la CNN inicial con MobileNetV2. La tabla resume la mejora global y la recuperación de la clase metal. La última ejecución de MobileNetV2 identificó 43 de las 47 imágenes de metal; su matriz de confusión permite consultar los errores por categoría.



(a) CNN inicial: 69,00 % en test.

(b) MobileNetV2: 95,53 % en test.

Figura 6: Matrices de confusión de la CNN inicial y del modelo con mejor resultado, calculadas sobre el conjunto de test.

Tabla 6: Evolución de los resultados de test entre la primera CNN, la quinta prueba manual y MobileNetV2.

Métrica	Prueba 1	Prueba 5	MobileNetV2
Accuracy	69,00 %	75,14 %	**95,53 %**
Aciertos / 358	247	269	**342**
Errores / 358	111	89	**16**
Recall metal	38,30 % (18/47)	48,94 % (23/47)	**91,49 % (43/47)**

En conjunto, los resultados muestran que ampliar la CNN desde cero mejora la clasificación, pero la transferencia de aprendizaje aporta el mayor salto en esta partición. MobileNetV2 reduce los errores totales de 111 en la prueba inicial a 16 y recupera mejor la clase metal. La evaluación se limita a las 358 imágenes del test disponible; no permite asegurar el mismo rendimiento en imágenes capturadas en otros entornos.

Referencias

[1] A. Paszke *et al.*, «PyTorch: An imperative style, high-performance deep learning library», en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.

[2] K. He, X. Zhang, S. Ren, y J. Sun, «Deep residual learning for image recognition», en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, y L.-C. Chen, «MobileNetV2: Inverted residuals and linear bottlenecks», en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, y M. Koyama, «Optuna: A next-generation hyperparameter optimization framework», en *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019.